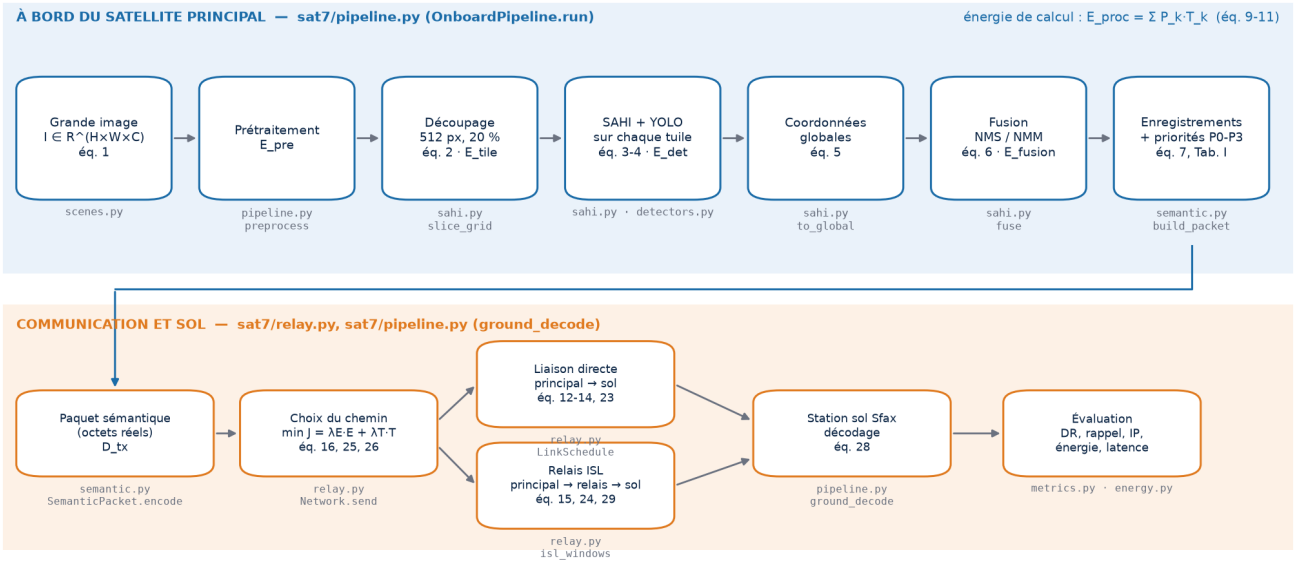
Guide du pipeline

Energy-Aware Semantic Downlink for Maritime Object Detection with Optional Inter-Satellite Relay

Chaque fichier, ce qu'il fait, comment les fichiers sont reliés, et faut-il ajouter un tableau de bord ?



Le pipeline du papier tel qu'il est implémenté : en haut, à bord du satellite ; en bas, communication et station sol. Sous chaque étape : le fichier et la fonction qui la réalisent.

ENIS · IASTAM 6.0 · Track 4, Problème 7 — 09/10/2026

1. En une page

Le dépôt contient **deux pipelines** qui partagent la même bibliothèque `code/sat7/` :

- **Le pipeline du papier** (nouveau, objet de ce guide) : grande image → tuiles 512 px avec 20 % de chevauchement → SAHI + YOLO → coordonnées globales → fusion → enregistrements sémantiques P0–P3 → liaison directe ou relais inter-satellite → station sol. Il est lancé par `scripts/paper_b0_b4.py`, qui exécute la campagne B0 → B4 du papier, ses 6 ablations et ses courbes de compromis.
- **Le pipeline historique « WP »** (wp0 à wp14) : il a produit les résultats **mesurés sur les vraies images Airbus** (557×, rappel 0,685, etc.). Il reste la référence pour les chiffres réels ; il est résumé en section 7.

Règle de lecture. Chaque équation du papier correspond à une fonction, et chaque fonction est vérifiée par un test dans `tests/test_paper.py`. Les résultats présentés ici sont étiquetés **SIM** : ils proviennent de scènes synthétiques et d'un détecteur classique de secours, car le dataset, les poids YOLO et l'environnement GPU ne sont pas sur cette machine. Ils montrent que la chaîne fonctionne ; ce ne sont pas des résultats YOLO.

Ce qui a été nettoyé

- Toute la chaîne embarquée est maintenant **un seul objet**, `OnboardPipeline` (`sat7/pipeline.py`). Ses étapes portent les noms des termes d'énergie de l'éq. 9 : `pre`, `tile`, `det`, `fusion`, `semantic`.
- Les configurations B0–B4 et les six ablations sont de simples valeurs de `PipelineConfig` déclarées dans `experiment_plan()` : il n'y a plus de variantes codées à la main dans le script.
- Le détecteur ne tourne qu'**une fois par découpage** (`detection_key`) ; les configurations qui ne diffèrent que par la fusion, le seuil ou la politique réutilisent ses détections.
- Le chemin mort `raw=` de `sahi_detect` a été supprimé ; le côté sol (`ground_decode`) et les centres livrés (`record_centres`) sont dans le même module que le côté bord.
- Un journal par paquet (`results/paper_packets_sim.csv`) est écrit : c'est la source de données du tableau de bord.
- Résultat identique avant et après nettoyage (mêmes rappels, octets, IP) ; 140 tests passent, 2 sont ignorés (ils demandent torch).
- Le pipeline WP est archivé dans `code/legacy_wp/` (modules, scripts, tests), toujours exécutable ; `code/sat7/` ne contient plus que le papier et le socle partagé ; les journaux d'exécution sont dans `results/logs/`.

2. Le pipeline du papier, étape par étape

Chaque ligne suit une donnée de gauche à droite : ce qui entre dans l'étape, ce qui en sort, et où c'est codé.

Étape (papier)	Équation	Fichier · fonction	Entrée → sortie
Grande image	éq. 1	scenes.py · synthetic_scene, airbus_mosaic	dataset → image H×W×3 + vérité terrain
Prétraitement	§III-B · E_pre	pipeline.py · preprocess	image brute → tableau uint8 contigu + T_pre
Découpage en tuiles	éq. 2 · E_tile	sahi.py · slice_grid	H, W → liste (x0, y0, 512, 512), pas = 512×(1−0,2)
Détection par tuile (SAHI)	éq. 3-4 · E_det	sahi.py · sahi_detect ; detectors.py · YoloDetector	tuiles → d = (x, y, w, h, p, c) locales
Coordonnées globales	éq. 5	sahi.py · Detection.to_global	x_g = x0 + x_l, y_g = y0 + y_l
Fusion des doublons	éq. 6 · E_fusion	sahi.py · fuse (NMS ou NMM, IoU ou IoS)	détections de tuiles voisines → D_global
Enregistrement sémantique	éq. 7	semantic.py · SemanticRecord	détection → {classe, confiance, bbox, horodatage, id image}
Priorités P0–P3	Tableau I	semantic.py · PriorityPolicy, build_packet	confiance → rejet / métadonnées / + ROI / + contexte
Paquet transmis	D_tx	semantic.py · SemanticPacket.encode	enregistrements + JPEG → octets réels
Choix du chemin	éq. 16, 25, 26	relay.py · Network.send, PathPolicy	paquet prêt à t → direct ou relais (min J, échéance)
Liaison directe	éq. 12-14, 23	relay.py · LinkSchedule ; orbit.py	passages SGP4 au-dessus de Sfax → E = P_tx·D/R
Relais inter-satellite	éq. 15, 24, 29	relay.py · isl_windows, relay_orbit	fenêtres ISL + passages du relais → E_ISL + E_GS
Décodage au sol	éq. 28	pipeline.py · ground_decode	octets → enregistrements + T_decoding
Évaluation	éq. 8-22, 27-30	metrics.py ; energy.py	rappel, F1, mAP, IP, E_total, ES, DR, T_total, objectif J

Les quatre objets qui circulent

- `Detection` (sahi.py) : une boîte d = (x, y, w, h, p, c). Tous les modules la partagent, c'est pourquoi sahi.py est au centre du graphe de dépendances.
- `SahiResult` : la sortie du détecteur sur une image, avec les détections brutes et les temps T_tile, T_det. Réutilisable par plusieurs configurations.
- `OnboardOutput` (pipeline.py) : les détections finales, le **paquet en octets** et le temps de chaque étape. C'est tout ce qui quitte le satellite.
- `PathChoice` (relay.py) : le chemin pris (direct, relais ou aucun), l'heure de livraison, E_comm et T_comm, plus les options écartées (pour l'éq. 35).

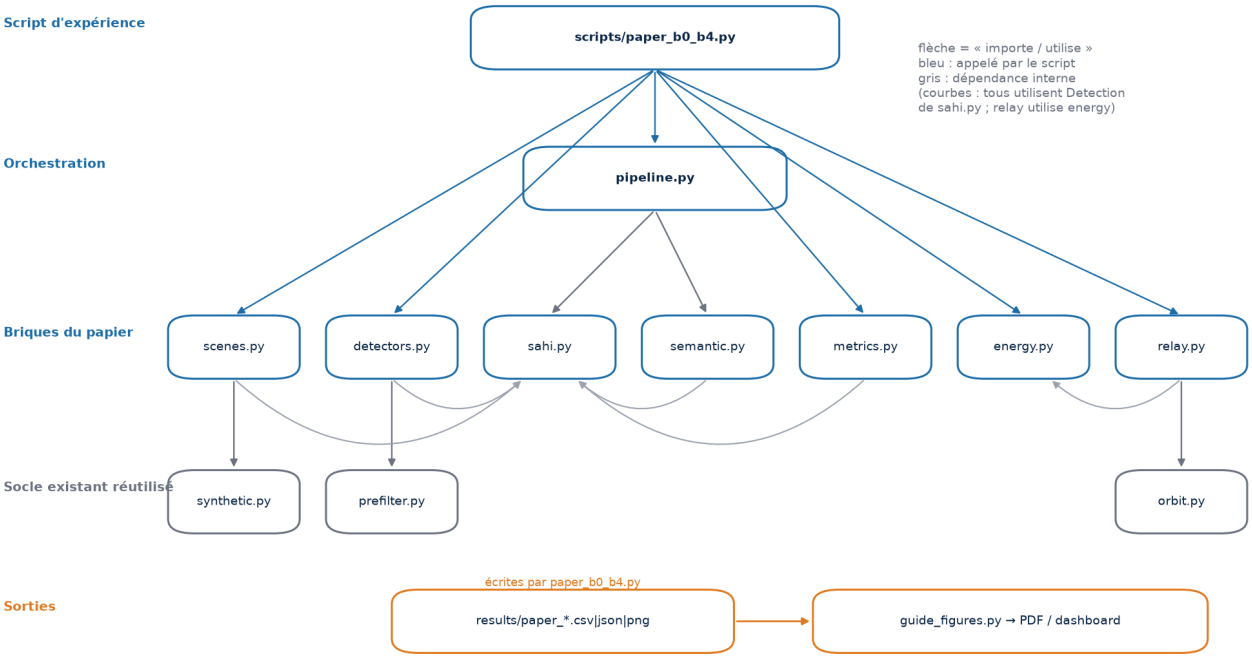
3. Chaque fichier du pipeline du papier

Fichier	Rôle	Contenu principal	Utilisé par
sat7/sahi.py	Découpage, SAHI, coordonnées globales, fusion (éq. 1-6)	Detection, SahiConfig (tile=512, overlap=0,20, fusion, seuil), slice_grid, fuse, sahi_detect, direct_detect	pipeline, detectors, semantic, metrics, scenes
sat7/detectors.py	Brancher un détecteur dans SAHI	YoloDetector (poids WP1, Ultralytics) ; ClassicDetector (secours sans torch, pré-filtre classique + confiance par contraste)	paper_b0_b4, guide_figures
sat7/semantic.py	Représentation sémantique et politique (éq. 7, Tab. I)	SemanticRecord (24 octets, +8 si géo), PriorityPolicy (t_discard 0,25 / t_context 0,40 / t_meta 0,70), RoiConfig, build_packet, SemanticPacket.encode/decode	pipeline
sat7/pipeline.py	Orchestration de bord + décodage sol	PipelineConfig, OnboardPipeline (detect, detections, run, with_threshold), OnboardOutput, preprocess, ground_decode, record_centres	paper_b0_b4, tests
sat7/energy.py	Modèle énergie / latence / objectif (éq. 8-22, 27-30)	PowerConfig (HYPOTHÈSES), processing_energy, comm_energy_direct/relay, energy_saving, downlink_reduction, total_latency, select_operating_point	relay, paper_b0_b4
sat7/relay.py	Liaisons et choix du chemin (éq. 15-16, 23-26, 29)	Window, LinkSchedule (FIFO sur fenêtres), ground_windows, isl_windows, RelayConfig (HYPOTHÈSES), PathPolicy, Network.send	paper_b0_b4, guide_figures
sat7/metrics.py	Métriques §VIII-E	match (IoU ≥ 0,5), DetAccumulator (précision, rappel, F1, AP, rappel des petits objets < 32 px), preserved (préservation de l'information)	paper_b0_b4
sat7/scenes.py	Grandes scènes (éq. 1)	airbus_mosaic (2×2 vraies tuiles Airbus = 1536 px, RÉEL) ; synthetic_scene (SIM)	paper_b0_b4, guide_figures
scripts/paper_b0_b4.py	La campagne du papier (§VIII)	experiment_plan (B0-B4 + ablations), boucle de bord, transmit (réseau), summarise, balayage du seuil, objectif éq. 20, sorties CSV/JSON/PNG	—
scripts/guide_figures.py	Figures de ce guide	schémas, graphe des fichiers, maquettes du tableau de bord	build_pipeline_guide
scripts/build_pipeline_guide.py	Ce PDF	mise en page reportlab	—
tests/test_paper.py	Une vérification par équation	16 tests : tuiles, éq. 5, fusion, aller-retour du paquet, Tableau I, énergie, objectif, liaisons, relais, métriques, chaîne complète	pytest

Fichiers existants réutilisés tels quels

- `sat7/orbit.py` : orbite SGP4, station Sfax, capacité de chaque passage (Shannon). Utilisé par relay.py pour les passages du satellite principal et du relais.
- `sat7/prefilter.py` : filtre classique (Otsu, morphologie). Utilisé par le détecteur de secours.
- `sat7/synthetic.py`, `sat7/rle.py` : scènes synthétiques et boîtes.

4. Les relations entre les fichiers



Graphe des dépendances du pipeline du papier. Une flèche signifie « importe et utilise ». Il est extrait des imports réels du code.

Comment lire ce graphe

- **Un seul point d'entrée** : `paper_b0_b4.py` construit les scènes, choisit le détecteur, crée un OnboardPipeline par configuration, puis envoie les paquets dans le réseau et calcule les métriques.
- **pipeline.py ne dépend que de sahi.py et semantic.py** : la partie à bord ne sait rien de l'orbite ni de l'énergie. C'est voulu : c'est le code qui irait sur le satellite.
- **La communication est séparée** : relay.py (avec orbit.py et energy.py) ne reçoit que des tailles de paquets et des heures. Le relais ne voit jamais l'image, comme l'exige le papier.
- **Les sorties sont des fichiers** dans code/results/ : le PDF et le futur tableau de bord les *lisent*, ils ne relancent jamais le calcul.

Le déroulement d'une exécution

#	Ce qui se passe	Où
1	Pour chaque scène : prétraitement, puis le détecteur une fois par découpage (512/20 %, 512/0 %, image entière)	paper_b0_b4.main → OnboardPipeline.detect
2	Pour chaque configuration : seuil + fusion + paquet, mesure de chaque temps	OnboardPipeline.run
3	Décodage du paquet comme au sol, centres des enregistrements livrés	ground_decode, record_centres
4	Fenêtres de liaison sur 48 h : passages principal, ISL, passages relais	ground_windows, isl_windows
5	Paquets envoyés dans l'ordre de capture, chemin choisi par min J sous échéance	transmit → Network.send
6	Métriques, énergie, objectif, CSV + JSON + figure	summarise, select_operating_point

5. Campagne B0 → B4 et ablations

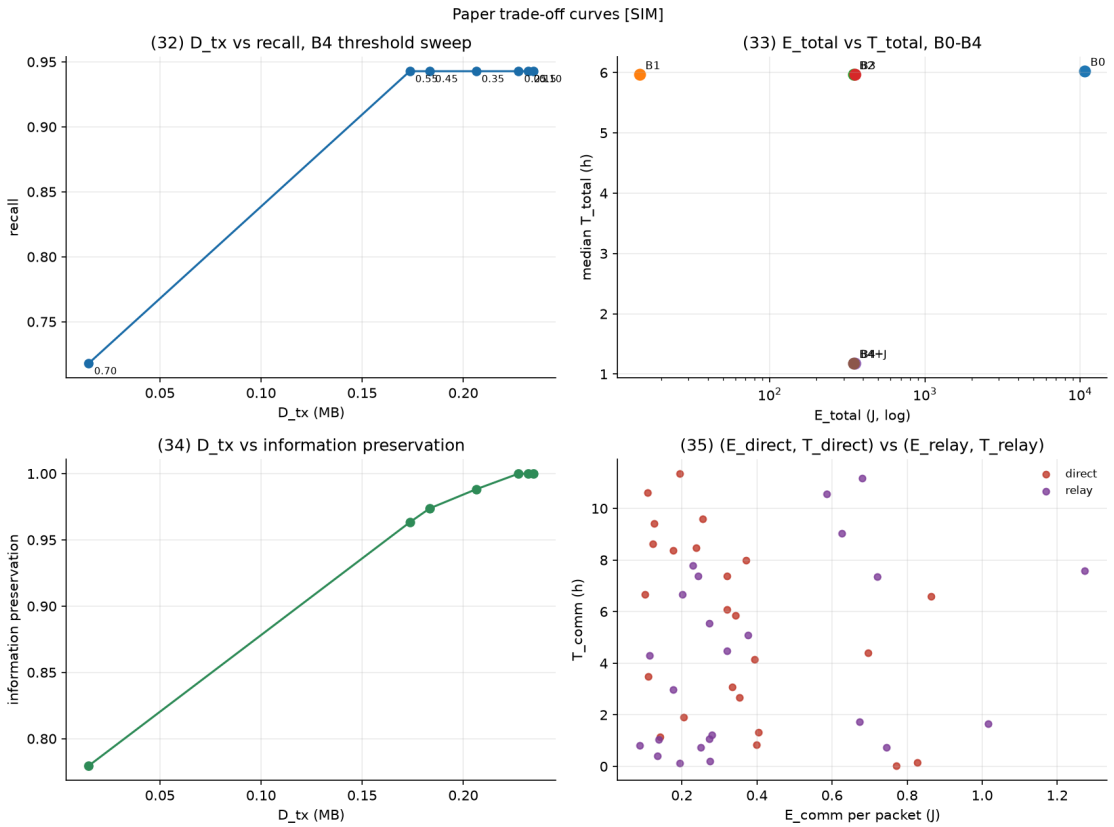
Chaque configuration du Tableau II du papier est une valeur de PipelineConfig ; les ablations du §VIII-G changent un seul champ par rapport à B4.

Config.	Ce qui est fait à bord	Ce qui est envoyé	PipelineConfig
B0	rien	image brute	— (octets bruts)
B1	YOLO sur l'image entière	sortie de détection	sahi=False, semantic=False
B2	SAHI + YOLO	sortie de détection	semantic=False
B3	SAHI + YOLO + politique	métadonnées / ROI, direct	défaut, relais interdit
B4	SAHI + YOLO + politique	direct ou relais	défaut, relais permis

Résultats [SIM] — 24 scènes de 2048 × 2048

Config.	D_tx (Mo)	Réduction	Rappel	F1	IP	E_total (J)	ES	Latence méd.
B0	302	×1	0,94	0,75	1,00	10 733	0,0%	6,02 h
B1	0.0178	×17 004	0,67	0,55	0,54	15	99,9%	5,97 h
B2	0.0188	×16 091	0,94	0,75	1,00	348	96,8%	5,97 h
B3	0.227	×1 328	0,94	0,75	1,00	356	96,7%	5,97 h
B4	0.227	×1 328	0,94	0,75	1,00	356	96,7%	1,18 h
B4+J	0.0188	×16 029	0,94	0,75	1,00	348	96,8%	1,18 h

- **SAHI compte** : rappel 0,67 sans découpage contre 0,94 avec, pour environ 25× plus d'énergie de calcul.
- **Le relais divise la latence** : 5,97 h en direct seul (B3) contre 1,18 h (B4) ; 42% des paquets passent par l'ISL.
- **Ablations** : sans fusion, la précision tombe de 0,63 à 0,40 (doublons) ; sans politique adaptative, 4.3× plus d'octets.
- **Objectif éq. 20** (F1 ≥ 0,5, IP ≥ 0,9) : meilleur seuil 0,55.



Courbes de compromis du papier (éq. 32 à 35), produites par paper_b0_b4.py [SIM].

6. Mesuré, supposé, et comment l'exécuter

Mesuré	Supposé (affiché à chaque exécution)
détections ; octets de chaque paquet (sérialisation binaire réelle et JPEG réels) ; temps de chaque étape T_k sur la machine ; passages au-dessus de Sfax et géométrie ISL (SGP4)	puissances P_k (PowerConfig) ; débits des liaisons et de l'ISL ; orbite du relais (RAAN +60°) ; heures de capture ; échéance 6 h ; poids λ_E, λ_T ; ciel des scènes synthétiques

- Les temps viennent d'un ordinateur portable, pas du matériel de vol : l'énergie est une **estimation**.
- Le papier cite xView, VisDrone, DOTA et HRSID ; le code utilise **Airbus** (optique), le seul jeu de données préparé. Les grandes scènes réelles sont des mosaïques 2×2 : les jointures entre tuiles sont artificielles.

Commandes

```
cd code
# tests (140 réussis, 2 ignorés sans torch)
.venv/Scripts/python.exe -m pytest tests legacy_wp/tests -q

# campagne SIM, n'importe où
.venv/Scripts/python.exe scripts/paper_b0_b4.py

# campagne RÉELLE : machine GPU avec data/ et runs/
.venv312/Scripts/python.exe scripts/paper_b0_b4.py --source airbus \
    --detector yolo --scenes 200

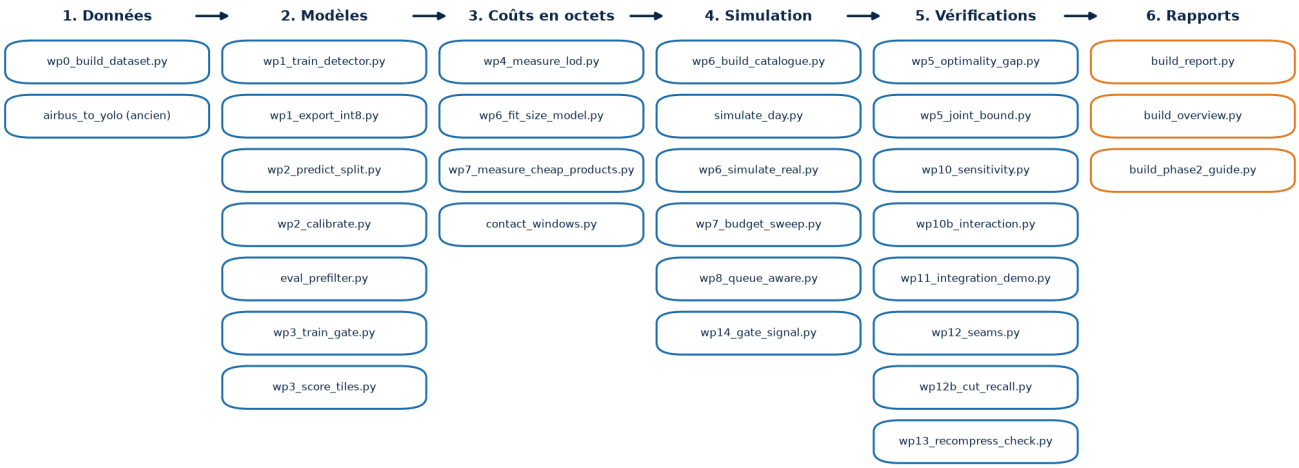
# ce guide
.venv/Scripts/python.exe scripts/guide_figures.py
python scripts/build_pipeline_guide.py
```

Tous les paramètres du §VIII-D sont des options : --tile, --overlap, --fusion, --conf, --link, --deadline-hours, plus PowerConfig et RelayConfig dans le code.

7. Le pipeline historique « WP » (résultats réels)

Emplacement : `code/legacy_wp/` (archivé, toujours exécutable), sauf wp0, wp1 et wp2, restés dans `code/scripts/` car le pipeline du papier en a besoin.

Avant l'implémentation du papier, le projet a été construit en lots de travail (WP). Ces scripts ont produit tous les chiffres **mesurés sur Airbus** du README et des rapports. Ils partagent sat7/ avec le pipeline du papier mais ne l'appellent pas.



Les scripts WP, de gauche à droite dans l'ordre où leurs sorties s'enchaînent.

Script	Ce qu'il fait
wp0_build_dataset	dataset Airbus sans fuite (doublons regroupés, sat7/dedup)
wp1_train_detector · wp1_export_int8	entraîne YOLOv8n (mAP50 0,804) ; export ONNX FP32/INT8
wp2_predict_split · wp2_calibrate	prédictions sur val ; calibration de la confiance
eval_prefilter · wp3_train_gate · wp3_score_tiles	filtre classique (échec : 0,645) ; petit CNN « gate » (0,988 en 0,15 ms)
wp4_measure_lod · wp6_fit_size_model · wp7_measure_cheap_products	taille réelle de chaque niveau de détail ; loi octets = a·taille^b
contact_windows	passages au-dessus de Sfax et capacité
wp6_build_catalogue · wp6_simulate_real	catalogue des tuiles réelles ; résultat principal 557×, rappel 0,685
simulate_day	première version sur journée synthétique
wp7_budget_sweep · wp8_queue_aware · wp14_gate_signal	où vont les octets ; adaptation à la file ; gate appris pour les vignettes
wp5_optimality_gap · wp5_joint_bound	glouton vs optimum exact
wp10_sensitivity · wp10b_interaction	les hypothèses balayées (80 réglages)
wp11_integration_demo	chaîne complète sur vraies images JPEG
wp12_seams · wp12b_cut_recall · wp13_recompress_check	bords de tuiles et SAHI mesurés ; recompression côtière
build_report · build_overview · build_phase2_guide	les PDF du dossier paper/

Relation entre les deux pipelines. Le pipeline WP a établi les chiffres réels (détecteur, coûts en octets, ordonnanceur). Le pipeline du papier reprend son détecteur (YoloDetector charge les poids de wp1), son orbite (orbit.py) et ses mesures de SAHI (wp12) comme point de comparaison. Ses résultats deviendront RÉELS quand paper_b0_b4.py sera lancé avec --source airbus --detector yolo.

8. Faut-il ajouter un tableau de bord ?

Recommandation : oui, mais au sol et en lecture seule. Un tableau de bord qui lit les fichiers de code/results/ vaut l'effort. Il ne faut **pas** l'ajouter au pipeline embarqué : rien ne s'affiche à bord d'un satellite, et le papier ne le demande pas.

Pourquoi oui

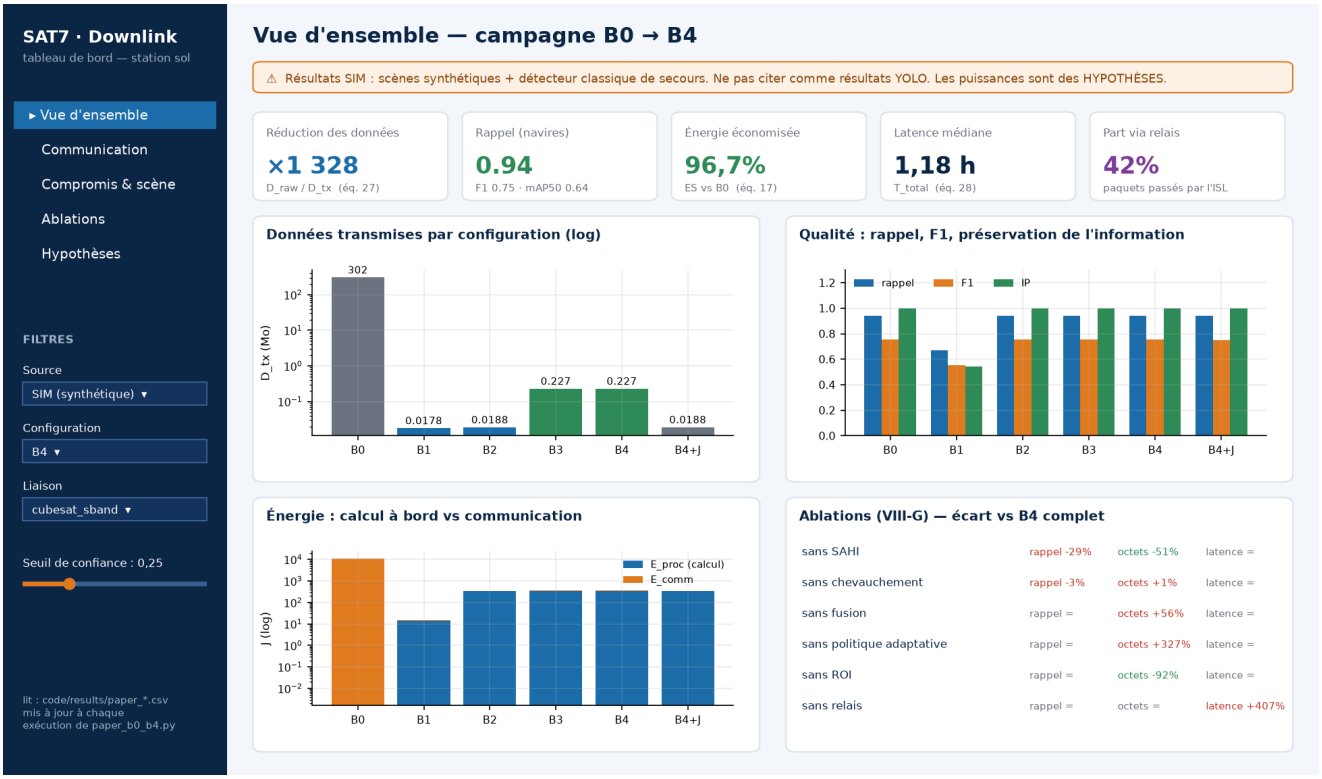
- **Pour le jury et la présentation** : le papier repose sur des compromis (octets contre rappel, énergie contre latence, direct contre relais). Un curseur de seuil qui fait bouger les courbes se comprend en dix secondes ; huit tableaux de chiffres, non.
- **Pour l'équipe** : voir d'un coup d'œil si une nouvelle exécution est SIM ou RÉELLE, si les paquets respectent l'échéance, et quelle ablation coûte le plus.
- **Coût faible** : toutes les données existent déjà (paper_b0_b4_sim.csv, paper_ablations_sim.csv, paper_tradeoff_sim.csv, paper_packets_sim.csv). Le tableau de bord ne recalcule rien. Compter un à deux jours.

Les pièges à éviter

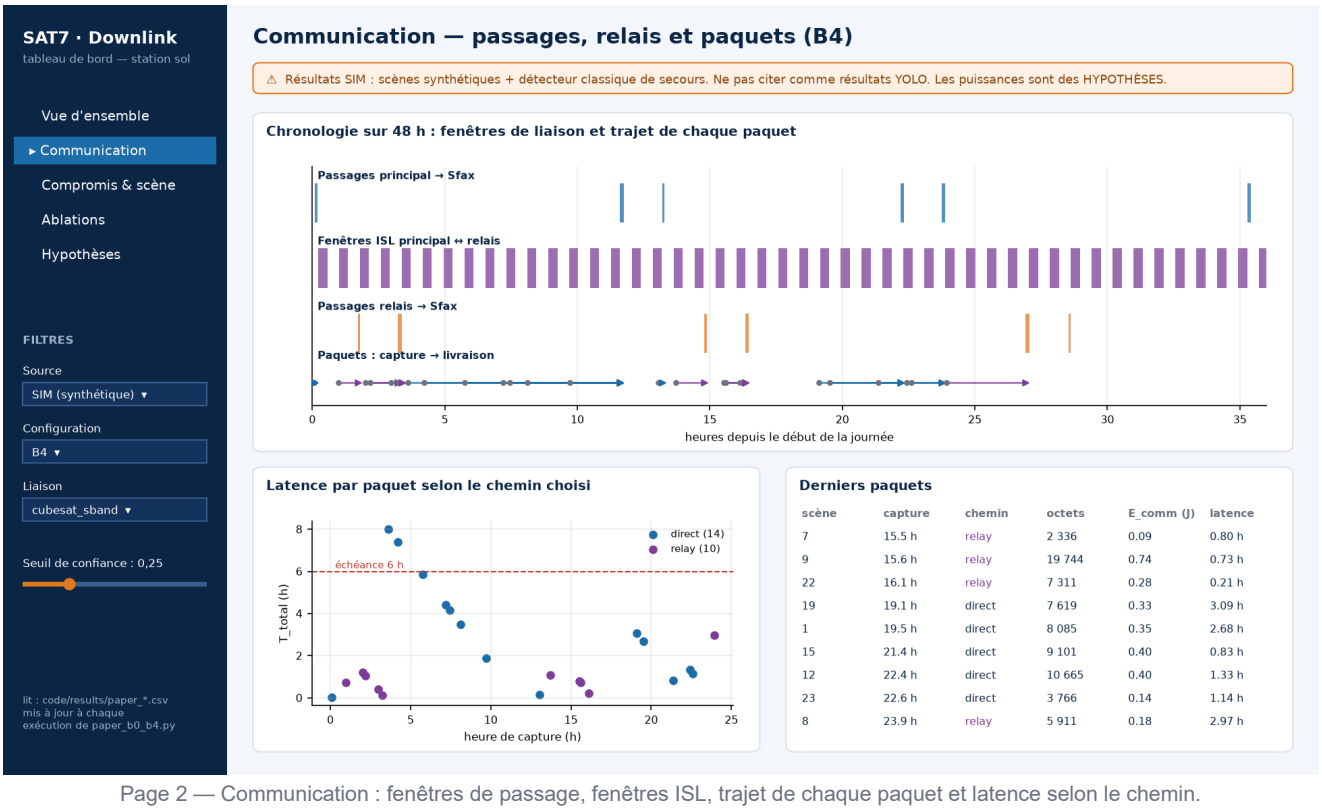
- **Ne jamais mélanger SIM et RÉEL** : bandeau permanent indiquant la source, comme sur les maquettes ci-dessous.
- **Ne pas recalculer dans le tableau de bord** : il lit les CSV ; pour de nouveaux chiffres, relancer paper_b0_b4.py. Sinon deux versions des résultats coexistent.
- **Ne pas cacher les hypothèses** : une page « Hypothèses » liste les puissances, débits et l'orbite du relais lus dans paper_b0_b4_sim.json.

Maquettes

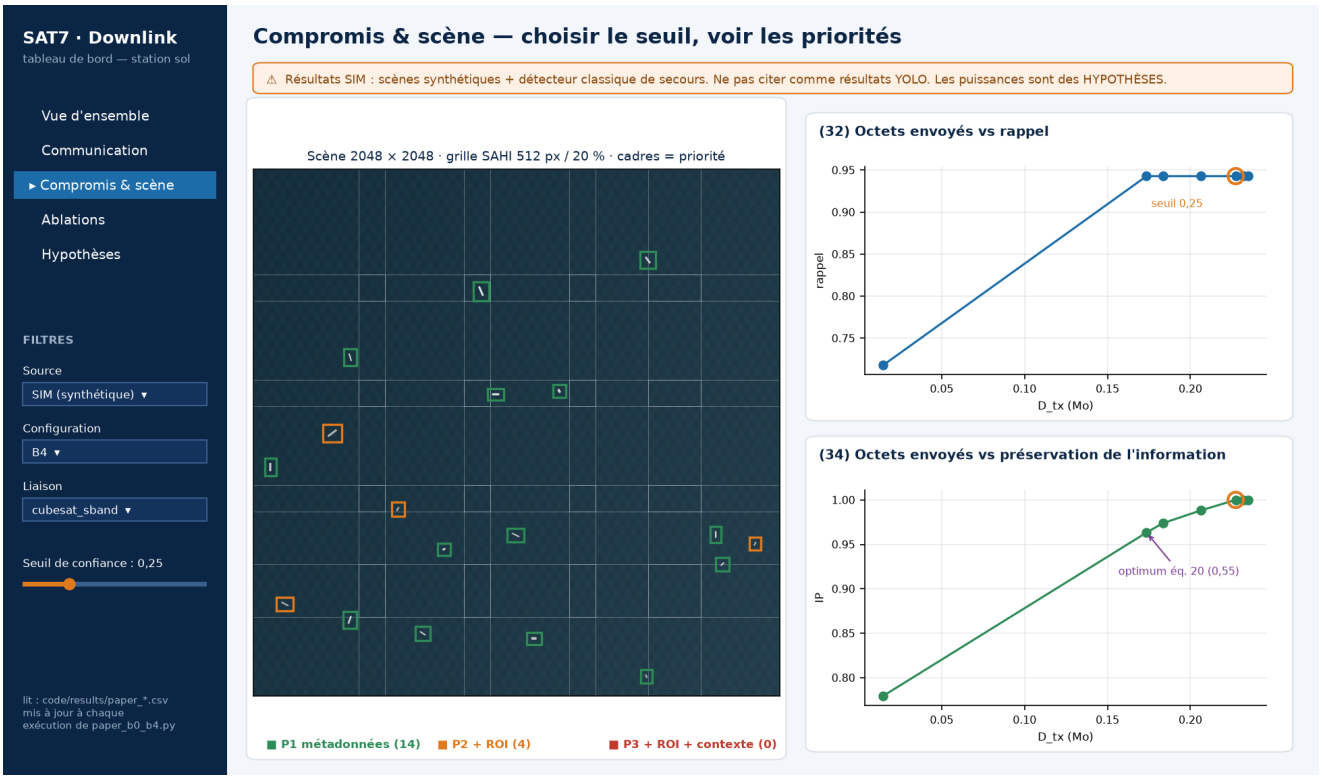
Les trois pages ci-dessous sont des **maquettes** dessinées avec les vraies sorties de la campagne SIM (mêmes chiffres que la section 5) et la vraie géométrie des liaisons. Elles montrent ce que le tableau de bord afficherait ; elles ne sont pas des captures d'une application déjà construite.



Page 1 — Vue d'ensemble : indicateurs clés de B4, octets transmis, qualité, énergie, effet de chaque ablation.



Page 2 — Communication : fenêtres de passage, fenêtres ISL, trajet de chaque paquet et latence selon le chemin.



Page 3 — Compromis & scène : la scène avec la grille SAHI et les priorités P1/P2/P3, et les courbes éq. 32 et 34 au seuil choisi.

9. Comment le construire

Outil conseillé

Streamlit + Plotly, en Python. Le projet est déjà en Python et pandas lit les CSV ; Streamlit fournit les menus, curseurs et onglets sans écrire de HTML, et Plotly rend les graphiques interactifs (survol, zoom). Alternative sans serveur : une page HTML statique générée par un script, plus simple à partager mais sans curseur réel.

Architecture

Élément	Contenu
code/dashboard/app.py	point d'entrée : barre latérale (source, configuration, seuil) et navigation
code/dashboard/data.py	chargement des CSV et du JSON, avec cache ; choix SIM (_sim) ou RÉEL (sans suffixe) selon les fichiers présents
code/dashboard/pages/	1_vue_ensemble.py, 2_communication.py, 3_compromis.py, 4_ablations.py, 5_hypotheses.py
Sources	paper_b0_b4*.csv, paper_ablations*.csv, paper_tradeoff*.csv, paper_packets*.csv, paper_b0_b4*.json

Étapes

- 1. Installer : pip install streamlit plotly (dans .venv).
- 2. Écrire data.py : une fonction par fichier, décorée de @st.cache_data.
- 3. Page Vue d'ensemble : st.metric pour les 5 indicateurs, deux graphiques en barres.
- 4. Page Communication : chronologie (barres horizontales des fenêtres et segments capture → livraison depuis paper_packets), nuage de latence, tableau des paquets.
- 5. Page Compromis : st.select_slider sur les seuils du balayage ; marquer le point choisi sur les courbes éq. 32 et 34 ; afficher l'optimum de l'éq. 20 lu dans le JSON.
- 6. Bandeau SIM/RÉEL et page Hypothèses lus dans le JSON.
- 7. Lancer : streamlit run dashboard/app.py, puis ouvrir http://localhost:8501.

Squelette de départ (non exécuté sur cette machine : Streamlit n'y est pas installé)

```
# code/dashboard/app.py
import json
from pathlib import Path
import pandas as pd
import plotly.express as px
import streamlit as st

RES = Path(__file__).resolve().parents[1] / "results"

@st.cache_data
def load(tag: str):
    read = lambda n: pd.read_csv(RES / f"{n}{tag}.csv")
    meta = json.loads((RES / f"paper_b0_b4{tag}.json").read_text())
    return (read("paper_b0_b4"), read("paper_ablations"),
            read("paper_tradeoff"), read("paper_packets"), meta)

st.set_page_config("SAT7 Downlink", layout="wide")
real = (RES / "paper_b0_b4.csv").exists()
source = st.sidebar.radio("Source", ["RÉEL", "SIM"] if real else ["SIM"])
tab, abl, sw, pk, meta = load("" if source == "RÉEL" else "_sim")
if meta["label"] == "SIM":
    st.warning("Résultats SIM : scènes synthétiques, détecteur de secours.")

b4 = tab.set_index("config").loc["B4"]
c = st.columns(5)
c[0].metric("Réduction", f"x{b4.reduction_x:,.0f}")
c[1].metric("Rappel", f"{b4.recall:.2f}")
c[2].metric("Énergie économisée", f"{b4.ES:.1%}")
c[3].metric("Latence médiane", f"{b4.T_total_med_h:.2f} h")
c[4].metric("Part relais", f"{b4.relay_share:.0%}")

left, right = st.columns(2)
left.plotly_chart(px.bar(tab, x="config", y="D_tx_MB", log_y=True))
thr = st.sidebar.select_slider("Seuil", options=list(sw.det_thr), value=0.25)
```

```
fig = px.line(sw, x="D_tx_MB", y="recall", markers=True)
sel = sw[sw.det_thr == thr]
fig.add_scatter(x=sel.D_tx_MB, y=sel.recall, mode="markers",
                marker=dict(size=16, color="orange"), name=f"seuil {thr}")
right.plotly_chart(fig)
st.dataframe(pk[["scene", "t_capture_h", "path", "bytes", "T_total"]])
```

Ce squelette couvre la page Vue d'ensemble et le curseur de seuil ; les autres pages suivent le même modèle. Il faudra le tester une fois Streamlit installé.